

*Corso di Laurea in Informatica A.A. 2024-2025*

---

# Laboratorio di Sistemi Operativi

Alessandra Rossi

---

---

# Il linguaggio C

---

I sorgenti devono essere creati utilizzando un editor di testo (vi/emacs...)

Non utilizzare word processor!

Per gcc, il suffisso del nome del file identifica la/le operazione/i che il compilatore esegue:

<file>.c: Codice sorgente C che deve essere preprocessato

<file>.C (maiuscola): Codice sorgente C++.

<file>.h: Header file che non deve essere ne' compilato ne' utilizzato per l'operazione di link

---

# I file C

---

I file “.c” contengono:

- direttive per il pre-processore
- codice sorgente

I file “.h” possono contenere:

- direttive per il pre-processore
- dichiarazioni di funzioni
- dichiarazioni di variabili, strutture

I file “.h” NON contengono l'implementazione delle funzioni



---

# I file C

---

E' opportuno distribuire le funzionalita' delle applicazioni complesse in piu' file.

Semplifica lo sviluppo, il debugging ed il riuso del codice.

L'accorpamento delle funzionalita' avviene durante la compilazione:

NON utilizzare direttive del tipo:

`#include "miofile.c"`

---

# Compilare C

---

Nei sistemi Linux, la compilazione di sorgenti C avviene utilizzando il comando gcc (GNU C Compiler)

Per compilare il programma test1.c potrebbe essere sufficiente eseguire:

```
gcc test1.c
```

Il nome dell'eseguibile, in questo caso, è a.out

L'opzione -Wall mostra tutti i warning

---

# Compilare C

---

Per creare l'eseguibile, gcc attraversa varie fasi, tra cui:

- preprocessing

- compilazione

- linking

Se l'applicazione e' distribuita in piu' sorgenti, puo' essere sufficiente eseguire il seguente comando:

```
gcc -o nomefile file1.c file2.c...
```

In questo caso, il nome del programma eseguibile e' nomefile



---

# Compilare C

---

Normalmente, il compilatore esegue sia la compilazione che il linking

**gcc -c file1.c** esegue solo la compilazione, producendo il file oggetto file1.o

se abbiamo i file oggetto file1.o e file2.o, possiamo “linkarli” con **gcc file1.o file2.o**, producendo a.out

il programma **make** automatizza questo processo

---

# Funzioni

---

In ogni applicazione esiste sempre un'unica funzione main  
unica per l'applicazione  
NON “unica in ogni file”.

Le “firme” standard della funzione main sono:

`int main(int argc, char *argv[])`

Consente di ottenere i parametri passati all'applicazione dalla linea di comando

`int main(void)`



---

# I delimitatori

---

Il terminatore o delimitatore di istruzioni e' il “;”

Oppure la “,” ma con un significato specifico

Le parentesi { } delimitano un blocco di istruzioni

E' necessario delimitare i blocchi contenenti almeno due istruzioni

E' opzionale delimitare i blocchi contenenti un'unica istruzione (es. “if”)

---

# Le variabili

---

Tutte le variabili devono essere dichiarate prima del loro uso

La variabile e' visibile solo all'interno del blocco in cui e' dichiarata

Il C fornisce i seguenti tipi di dato base:

char, int, float, double, long, short.

NON esiste il tipo "stringa"

E' possibile costruire tipi di dato complessi tramite il costrutto struct

E' possibile dichiarare puntatori a variabili

---

# Dichiarazioni

---

```
int a,b,c;
```

```
float d=0.0,e,f=1.0;
```

```
char g='a';
```

```
struct{
```

```
    int data;
```

```
    float ratio;
```

```
    char iniziale;
```

```
} struttura;
```

```
struttura.data=5;
```

```
struttura.ratio=1.0;
```

```
int array1[10];
```

```
int array2[10][10];
```

Costanti simboliche:

```
#define MINIMO 1
```

```
#define MASSIMO 5
```



---

# Le variabili

---

Il passaggio dei parametri puo' avvenire:

Per valore: eventuali modifiche all'interno della funzione NON hanno effetto al suo esterno

es., `int fun(int a);`

Per riferimento: Eventuali modifiche all'interno della funzione hanno effetto sul valore della variabile nella funzione chiamante

es. `int fun(int *a);`

---

# Le funzioni

---

E' opportuno **dichiarare** sempre le funzioni utilizzate all'interno di ogni file.

- semplifica la correzione degli errori legati ai tipi
- possibilmente tramite un file header “.h”

La firma (o prototipo) di una funzione include:

- tipo del valore restituito dalla funzione
- nome della funzione
- elenco dei parametri con rispettivi tipi

---

# Esempio

---

in pippo.h:

```
int fun(int a, float b);  
...
```

in pippo.c:

```
#include "pippo.h"  
  
int fun(int a, float b){  
    return(a+(int)b);  
  
}
```



---

# Esempio

---

Qual e' il comportamento del seguente codice C ?

```
#include <stdio.h>
#include <unistd.h>

int main(void)
{

    printf("Hello World!");
    sleep(5);
}
```

Una volta in esecuzione, aspetta 5 secondi e POI stampa “Hello World!”

---

# I cicli

---

```
If (condizione){  
    lista istruzioni  
}
```

```
else{  
    lista istruzioni  
}
```

```
while (condizione){  
    lista istruzioni  
}
```

```
for (expr1;expr2;expr3){  
    lista istruzioni  
}
```

```
equivalente a:  
expr1;  
while (expr2){  
    lista istruzioni  
    expr3;  
}
```



---

# Gli operatori

---

Operatori relazionali:

> >= < <= == !=

Operatori logici

&& (and) || (or) ! (not)

Operatori di incremento e decremento:

k++ ++k k-- --k

Operatori orientati ai bit:

& (and) | (or) ^ (xor) << (shift a sx)

>> (shift a destra) ~ (complemento a uno)

---

# I puntatori

---

Un puntatore è una variabile che contiene l'indirizzo di memoria di un'altra variabile.

Quando dichiariamo una variabile, a questa verrà riservato un indirizzo di memoria, ad esempio la posizione 1000. Un puntatore contiene, appunto, l'indirizzo di tale variabile (quindi il valore 1000)

```
// variabile intera  
int variabile;
```

```
// puntatore ad un intero  
int *puntatore;
```

```
// assegno al puntatore l'indirizzo di variabile  
puntatore = &variabile;
```

# Esempio

```
#include <stdio.h>
int main()
{
    int alfa = 4;
    int beta = 7;
    int *pointer;
    pointer = &alfa;
    printf("alfa -> %d, beta -> %d, pointer -> %dn", alfa, beta, pointer);
    beta = *pointer;
    printf("alfa -> %d, beta -> %d, pointer -> %dn", alfa, beta, pointer);
    alfa = pointer;
    printf("alfa -> %d, beta -> %d, pointer -> %dn", alfa, beta, pointer);
    *pointer = 5;
    printf("alfa -> %d, beta -> %d, pointer -> %dn", alfa, beta, pointer);
}
```

alfa -> 5, beta -> 4, pointer -> 100



---

# I puntatori

---

```
#include <stdio.h> int main(void){
```

```
    int a=5;
```

```
    int *p;
```

```
    printf("%d",a);
```

```
    p=&a;
```

```
    *p=6; printf("%d",a);
```

p=&a; (tutte le operazioni su “\*p” sono operazioni su a)

p=a; ERRATO!

p accetta indirizzi di interi, NON interi!

```
}
```

---

# Il linguaggio C

---

Alcune funzioni di I/O standard utilizzano il *buffering*

utilizzano un'area di memoria per memorizzare le informazioni prima di inviarle al device  
riduce il numero di system call “effettive” e migliora le performance

Esistono tre tipi di buffering:

Completo: L'I/O effettivo avviene solo al riempimento del buffer

Non utilizzato per device interattivi come schermo e tastiera

Buffering a linea: L'I/O viene eseguito non appena viene inserito nel buffer un carattere newline '\n' (o al termine del processo)

Utilizzato da printf e scanf

Senza buffering: L'I/O avviene immediatamente

Utilizzato, ad esempio, per lo standard error

---

# Esempio

---

Qual e' l'output del seguente codice C ?

```
#include <stdio.h> int main(void)
{
    int x,y;
    x=0; y=0;
    while (x=y)
        x=x+1;
    printf("x=%d, y=%d\n",x,y) ;
}
```



---

# Esempio

---

Il seguente programma compilato termina con errore. Perché?

```
#include <stdio.h> int main(void)
{

    int x;
    printf("Inserisci un intero:");
    scanf("%d", x);
    printf("L'intero inserito è %d\n", x);
}
```

L'errore generato sui sistemi Unix è Segmentation Fault o, tentativo di accesso ad un “Segmento” di memoria non allocato. Questo tipo di errore è dovuto, in genere, all'uso improprio dei puntatori.

---

# Esempio

---

Qual e' l'errore in questo codice ?

```
#include <stdio.h> int main(void)
{

    int a[10], i;
    for(i=1; i<=10; i++)
        a[i] = i;
    printf("a[%d]=%d\n", i, a[i]);
}
```



---

# Esempio

---

Qual e' l'errore in questo codice ?

```
#include <stdio.h> #include <string.h>
int main(int argc, char *argv[])
{

    char *stringa;
    strcpy(stringa, argv[1]);
    printf("%s", stringa);
    return 0;
}
```

---

# Esempio

---

Qual e' l'errore in questo codice ?

```
#include <stdio.h> #include <string.h>
int main(int argc, char *argv[])
{
    char stringa[30];
    strcpy(stringa, argv[1]);
    printf("%s", stringa);
    return 0;
}
```

La dichiarazione `char *stringa` alloca lo spazio per un puntatore ad un carattere, NON per una stringa (array) di caratteri. Anche così, il programma non è molto corretto...

---

# Esempio

---

```
#include <stdio.h>

int main(void)
{
    int x,y;
    void swap(int a, int b);
    printf("Inserisci l'intero x=");
    scanf("%d", &x);
    printf("Inserisci l'intero y=");
    scanf("%d", &y);
    swap(x,y);
    printf("(y,x)=(%d,%d)\n",x,y);
    return 0;
```

Perche' la funzione non esegue lo swap ?

```
}
void swap(int a, int b)
{

    int temp; temp=a; a=b; b=temp; return
}
```



---

# Esempio

---

```
#include <stdio.h>
int main(void)
{
    int x,y;
    void swap(int *a, int *b);
    printf("Inserisci l'intero x=");
    scanf("%d", &x);
    printf("Inserisci l'intero y=");
    scanf("%d", &y);
    swap(&x,&y);
    printf("(y,x)=(%d,%d)\n",x,y);
    return 0;
```

Perche' la funzione non esegue lo swap ?

Le variabili passate per valore alla funzione swap, sebbene modificate, mantengono il loro varole originale.

```
}
void swap(int *a, int *b)
{

    int temp; temp=*a;
    *a=*b;
    *b=temp; return
}
```



*Università degli Studi di Napoli Federico II*

# THANK YOU FOR YOUR ATTENTION

TRUST ME ...  
I AM A ROBOT!

